

LeetCode 1296 - Divide Array in Sets of K Consecutive Numbers

1. Problem Understanding

We are given:

- An array `nums`
- An integer `k`

We have to determine whether the entire array can be divided into groups of size `k` such that:

- Every group contains exactly `k` consecutive numbers.
- Every element is used exactly once.

Example

```
nums = [1, 2, 3, 3, 4, 4, 5, 6]
```

```
k = 4
```

Possible groups

- `[1, 2, 3, 4]`
- `[3, 4, 5, 6]`

Answer

```
true
```

2. First Observation

Every group contains exactly `k` numbers.

Therefore,

if

```
nums.size() % k != 0
```

then

```
Answer = false
```

because one or more elements will always remain unused.

3. Why use a Map?

We need two things:

- Frequency of every number.
- Numbers in sorted order.

A map gives both.

Example

nums

1 2 3 3 4 4 5 6

Map becomes

Number	Frequency
1	1
2	1
3	2
4	2
5	1
6	1

Notice

The keys are automatically sorted.

4. The Greedy Observation (Most Important)

Always start with the smallest unused number.

Why?

Suppose

1 2 3 4

Can 1 belong to any other group except one starting from 1?

No.

It cannot be part of

2 3 4 5

or

```
3 4 5 6
```

because every group must be consecutive.

So the smallest unused number has only one choice:

It must be the first element of a group.

This is why the greedy strategy is correct.

5. What does freq mean?

Suppose

```
1 -> 2
2 -> 2
3 -> 2
```

The smallest number

```
1
```

appears

```
2
```

times.

So

```
freq = it->second;
```

means

```
freq = 2
```

This means

there are 2 copies of 1.

Can both copies go into the same group?

No.

Each group contains distinct consecutive numbers.

Therefore

those two copies must belong to two different groups.

So

freq tells us how many groups must start from the current smallest number.

6. Why check the next k numbers?

Suppose

```
1->2  
2->2  
3->2
```

Need

```
k=3
```

We must create

Group 1

1 2 3

Group 2

1 2 3

Therefore we need

- 1 -> 2 copies
- 2 -> 2 copies
- 3 -> 2 copies

The code checks exactly this.

```
if(count[it->first+i] < freq)  
    return false;
```

Meaning

If any required consecutive number has fewer than freq copies, forming the groups is impossible.

7. Why subtract freq?

This is the biggest doubt.

Many people think

"We only used one copy of 1."

That is incorrect.

Suppose

```
1->2  
2->2  
3->2
```

Since

```
freq=2
```

we are actually making

Group 1

1 2 3

Group 2

1 2 3

Both groups are formed together.

How many 2s are used?

```
2
```

How many 3s are used?

```
2
```

Therefore

```
count[it->first+i]-=freq;
```

is correct.

We consume

```
freq
```

copies from every consecutive number.

8. Why not subtract only 1?

Suppose

```
1->2  
2->2  
3->2
```

If we subtract only one,
Map becomes

```
1->1  
2->1  
3->1
```

This is wrong.
Because both copies of 1 were already used.
The correct map should become

```
1->0  
2->0  
3->0
```

Therefore subtraction must be

```
freq
```

not

```
1
```

9. What if the next number has smaller frequency?

Suppose

```
1->2  
2->2  
3->1
```

Need

```
freq=2
```

We must form

```
1 2 3  
1 2 3
```

Need

```
3 -> 2 copies
```

But only

```
1
```

exists.

Impossible.

Code

```
if(count[it->first+i]<freq)
    return false;
```

returns

```
false
```

immediately.

10. Are we actually building groups?

No.

This is another common doubt.

The checking solution never stores

Group 1

```
1 2 3
```

or

Group 2

```
1 2 3
```

It simply verifies

Can these groups exist?

using frequencies.

11. If we had to print the groups?

Then instead of subtracting

```
count[...] -= freq;
```

at once,

we print one group at a time.

Example

```
1->2
2->2
3->2
```

Output

```
1 2 3
1 2 3
```

Each printed group reduces frequency by

```
1
```

After printing two groups,
all frequencies become zero.
The greedy idea remains exactly the same.

12. Complete Algorithm

Step 1

If

```
n % k != 0
```

return

```
false
```

Step 2

Build the frequency map.

Step 3

Traverse the map in sorted order.

Step 4

For every number whose frequency is non-zero,

```
freq = frequency
```

This means

```
freq groups
```

must start from this number.

Step 5

Check

current

current+1

current+2

...

current+k-1

Each must have at least

```
freq
```

copies.

If not,

return

```
false
```

Step 6

Consume those numbers

```
count[current+i] -= freq;
```

Step 7

If every number is processed successfully,

return

```
true
```

13. Dry Run

Input

```
nums
```

1 2 3 3 4 4 5 6

```
k=4
```

Frequency

Number	Freq
1	1
2	1
3	2
4	2
5	1
6	1

Smallest

1

Frequency

1

Need

1 2 3 4

All exist.

Subtract

1->0
2->0
3->1
4->1

Next

2

Frequency

0

Skip.

Next

3

Frequency

```
1
```

Need

3 4 5 6

All exist.

Subtract

```
3->0
4->0
5->0
6->0
```

Everything becomes zero.

Return

```
true
```

14. Time Complexity

Building map

```
O(n log n)
```

Traversing and updating

```
O(n log n)
```

Overall

```
O(n log n)
```

Space

```
O(n)
```

FINAL INTUITION (REMEMBER THIS)

This is a Greedy + Frequency Map problem. We always process the smallest unused number because it has no alternative—it must start a consecutive group. If it appears `freq` times, then exactly `freq` groups must start from it. We simply verify that each of the next `k-1` consecutive numbers also has at least `freq` copies. If they do, we consume (`-freq`) those copies because they are now used in those groups. If any required number has fewer than `freq` copies, forming valid groups is impossible, so we return false. We never actually construct the groups in the checking solution; we only verify that they can exist using the frequency map.